

CanBot: Intelligent Waste Sorter

Process Report: EE297 — Intelligent Systems Project

STUDENT NAME — MD RAYHAN MIA | STUDENT ID — 23740371



**Maynooth
University**
National University
of Ireland Maynooth

MONDAY, 27 JANUARY 2025
DEPARTMENT OF ELECTRONIC ENGINEERING
MAYNOOTH UNIVERSITY

Contents

01. Introduction	02
1.1 Purpose of the Process Report	02
1.2 Overview of the Project	02
1.3 Team Roles	02
02. Project Planning	03
2.1 Idea Generation	03
2.3 Risk Assessment	03
03. System Design and Implementation	04
3.1 System Overview	04
3.2 Robotic Arm Design	04
3.3 Rover Base Development	04
3.4 Object Detection System	05
3.5 Hardware-Software Integration	05
3.6 Testing and Validation	
04. My Contribution	06
05. Challenges and Solutions	09
5.1 Technical Challenges	09
5.2 Non-Technical Challenges	10
06. Project Outcomes and Reflections	11
6.1 Project Outcomes	11
6.2 Reflections on the Project	11
6.3 Key Learnings	12
07. Conclusion	14
7.1 Summary of the Process	14
7.2 Future Recommendations	

01. Introduction

1.1 Purpose of the Process Report

This process report describes the plan and implementation phases of the project and reflects on the work done in collaboration within a team for the project objectives. It outlines the adopted strategies throughout the project, the challenges faced, and their solutions during the project life cycle.

1.2 Overview of the Project

This project focuses on designing and implementing a waste-sorting intelligent rover for detection, categorization, and sorting of bottles and cans into different bins. The integration of computer vision, robotics, and sustainable engineering practices in this work will contribute to the 12th United Nations Sustainable Development Goal of Responsible Consumption and Production. The hardware structure mainly includes the use of Raspberry Pi, YOLO v2, later replaced by YOLO v3, Arduino, and other hardware that enables real-time object detection and sorting.

1.3 Team Roles

The successful execution of this project required effective collaboration, with specific roles assigned to team members to ensure efficiency and clarity. Roles included:

Project Manager/Chairperson: James Fisher was the chairperson of this project. He oversaw project timelines and deliverables.

Robotic Arm Designers: Adam Hughes, Ales Vittek and Alexander Kiernan designed the robotic arm, and they also calculated & simulated the inverse kinematics for the arm.

Rover Base Developers: Daniel Zhurau, Md Rayhan Mia and Dharan Kumar Rajulapati were members of the rover base development team.

Computer Vision Developers: James Fisher & Md Rayhan Mia developed and trained the object detection model.

Integration Specialist: Ales Vittek managed hardware-software integration.

Documentation Team : All the group members worked on the technical report. Chibuikem Agu and Alexander Kiernan in charge of video log and PowerPoint.

This introduction provides the context for understanding how the team approached the project, and the methods used to achieve its goals.

02. Project Planning

2.1 Idea Generation

The project started with brainstorming on a meaningful and practical problem that can be solved with the use of embedded systems and intelligent devices. Guided by the 12th United Nations Sustainable Development Goal, which is Responsible Consumption and Production, the team came up with an autonomous rover for waste sorting. The thought was to identify, classify, and sort bottles and cans using computer vision and robotics, keeping in mind two important aspects: sustainability and automation.

2.2 Risk Assessment

Potential risks at the beginning of the project included planning for contingencies:

- Technical Risks:
 - Model performance is not as anticipated.
 - Integration of hardware and software may have some unexpected problems.
- Resource Risks:
 - Hardware component availability is low, or procurement takes a long time.
 - Insufficient labeled data to train the model.
- Time Management Risks:
 - Unforeseen technical challenges cause delays in reaching milestones.

Mitigation strategies included building buffer time into the timeline, regular team meetings to monitor progress, and preparing alternative plans (e.g., using pre-trained models if custom training failed).

This project planning phase ensured a structured approach, enabling the team to work efficiently, anticipate challenges, and maintain focus on the project objectives.

03. System Design and Implementation

Design and implementation of the waste-sorting rover system had several components combined that could meet the project's objectives. The system is made up of three major subsystems: robotic arm, rover base, and object detection system. In this section, we describe the design and implementation of each subsystem and the integration process.

3.1 System Overview

The rover system was designed to autonomously navigate, detect waste objects (bottles and cans), classify them, and sort them into designated bins. The key components included:

- A robotic arm for picking and sorting objects.
- A rover base equipped with wheels for mobility.
- A computer vision module powered by YOLO v3 for object detection and classification.

3.2 Robotic Arm Design

The arm design is done by Adam Hughes, Ales Vittek, and Alexander Kiernan by using SolidWorks and fusion 360. The design was meant for 6 degrees of freedom to allow flexibility in terms of object handling. Doing inverse kinematics calculations allows precise movement that enables the arm to pick up objects and place them into the correct bin with much ease. Servos and a gripper capable of handling weight and shape of both bottles and cans were fitted to the arm.

3.3 Rover Base Development

The base was designed by Daniel Zhurau, Md Rayhan Mia, and Dharan Kumar Rajulapati. MG996R Continuous Rotation Servo motors were set up for the forward-wheeled drive, while castors were used for the back to enable movement and rotation. The base was designed such that the robotic arm and onboard hardware, including a Raspberry Pi and an Intel NCS2, would move stably over any surface.

3.4 Object Detection System

First, object detection system YOLO v2 was used. Due to performance and accuracy issues, the team migrated to YOLO v3. The Computer Vision Team, James Fisher and Md Rayhan Mia, headed the collection and labeling of bottles and cans datasets, fine-tuning YOLO v3 to improve the detection and classification of bottles and cans. After updates, the model was deployed onto the Raspberry Pi with Intel NCS2 provided the computational power to run in real time.

3.5 Hardware-Software Integration

Hardware-software integration was overseen by Ales Vittek, who made sure everything worked in harmony. Integration tasks included:

- Synchronising object detection output with the robotic arm for precise object manipulation.
- Coordinating movement commands from the Raspberry Pi to the rover base and robotic arm.
- Managing power distribution across the system for consistent performance.

3.6 Testing and Validation

Each subsystem was tested in detail:

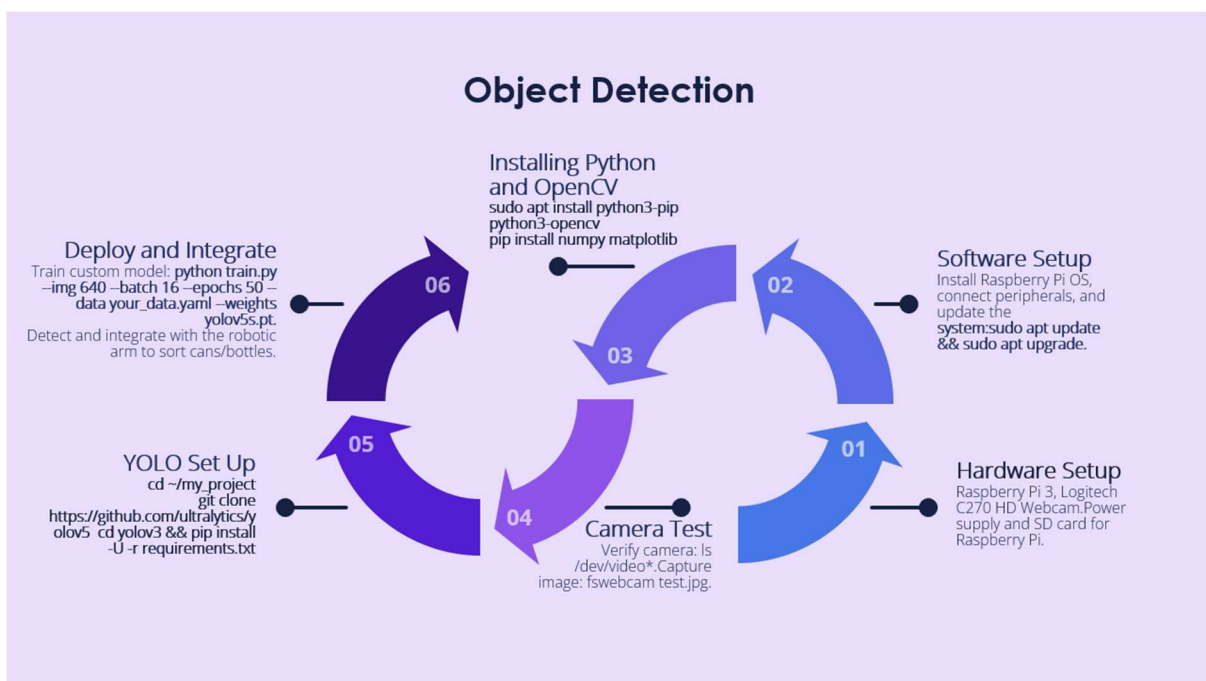
The robotic arm was tested with both simulated and real-world scenarios to ensure that it moved with precision. The base of the rover was tested for stability and navigation on different surfaces. The object detection system was tested for accuracy, speed, and reliability under various lighting conditions.

04. My Contribution: Object Detection Process with YOLO v3

During the project, YOLO v2 was chosen as the object detection model. However, when testing was running, some critical issues were found YOLO v2 was not getting good accuracy on different lighting conditions, occlusion cases. In addition, being optimised by Darknet software, its original implementation developed slower-than-desirable processing speeds for real-time applications. This hardly contributed to the overall reliability and responsiveness of the system.

To handle these issues, I recommend migration to YOLO v3, which provided multi-scale feature maps object detection more precisely and at higher speeds. In implementing YOLO v3, I ensure the detection system meets the requirements of the project for real-time and accurate object classification.

The steps that follow in carrying out the object detection using YOLO v3:



Step 1: Software Setup

First of all, the Raspberry Pi needed to be prepared for object detection. Installation of the OS was necessary, peripherals were connected, and an update was done using:

```
sudo apt update && sudo apt upgrade
```

This process established a stable and optimized software environment for subsequent implementation steps.

Step 2: Hardware Setup

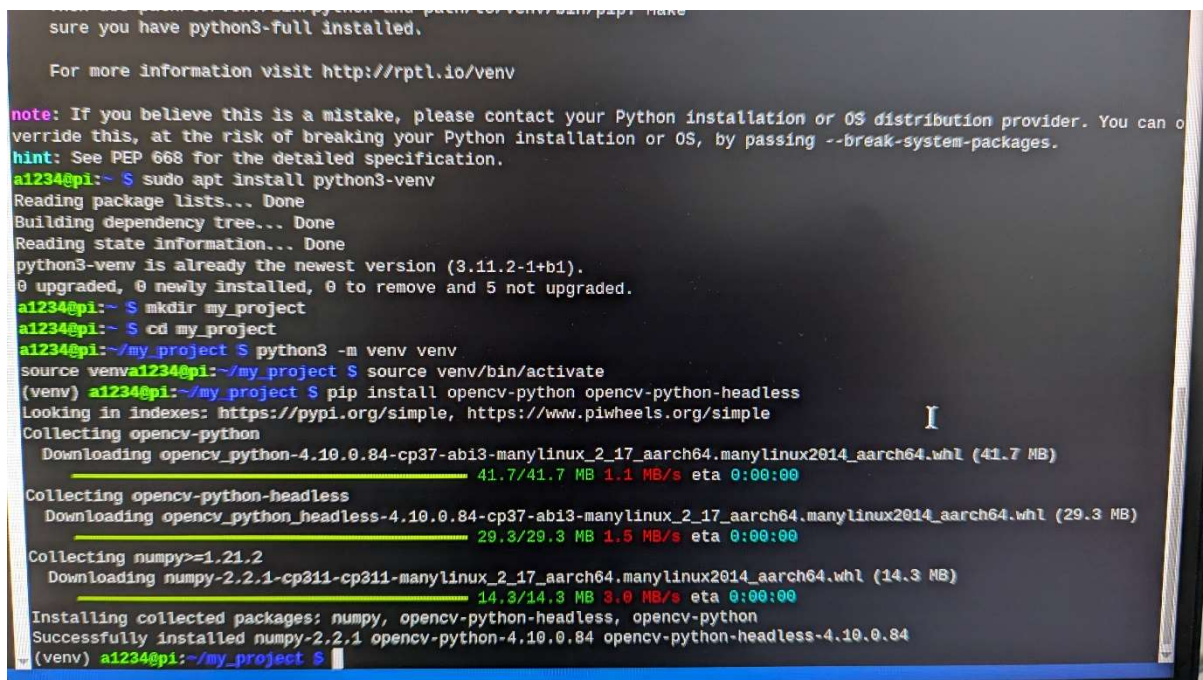
The hardware setup involved the configuration of the Raspberry Pi 3a, attachment of the Logitech C270 HD Webcam for capturing images, and preparation of the power supply and microSD card for the Raspberry Pi. These components ensure that the system can capture real-time video streams and process them effectively.

Step 3: Installing Python and OpenCV

To support image processing and object detection, I installed Python, along with essential libraries like OpenCV, NumPy, and Matplotlib, using the following commands:

```
sudo apt install python3-pip
pip install opencv-python numpy matplotlib
```

OpenCV was essential for camera interaction and preprocessing image frames before passing them to the YOLO model for detection.



```
note: If you believe this is a mistake, please contact your Python installation or OS distribution provider. You can o
verride this, at the risk of breaking your Python installation or OS, by passing --break-system-packages.
hint: See PEP 668 for the detailed specification.
a1234@pi:~$ sudo apt install python3-venv
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
python3-venv is already the newest version (3.11.2-1+b1).
0 upgraded, 0 newly installed, 0 to remove and 5 not upgraded.
a1234@pi:~$ mkdir my_project
a1234@pi:~$ cd my_project
a1234@pi:~/my_project$ python3 -m venv venv
source venva1234@pi:~/my_project$ source venv/bin/activate
(venv) a1234@pi:~/my_project$ pip install opencv-python opencv-python-headless
Looking in indexes: https://pypi.org/simple, https://www.piwheels.org/simple
Collecting opencv-python
  Downloading opencv_python-4.10.0.84-cp37-ab13-manylinux_2_17_aarch64.manylinux2014_aarch64.whl (41.7 MB)
    41.7/41.7 MB 1.1 MB/s eta 0:00:00
Collecting opencv-python-headless
  Downloading opencv_python_headless-4.10.0.84-cp37-ab13-manylinux_2_17_aarch64.manylinux2014_aarch64.whl (29.3 MB)
    29.3/29.3 MB 1.5 MB/s eta 0:00:00
Collecting numpy>=1.21.2
  Downloading numpy-2.2.1-cp311-cp311-manylinux_2_17_aarch64.manylinux2014_aarch64.whl (14.3 MB)
    14.3/14.3 MB 3.0 MB/s eta 0:00:00
Installing collected packages: numpy, opencv-python-headless, opencv-python
Successfully installed numpy-2.2.1 opencv-python-4.10.0.84 opencv-python-headless-4.10.0.84
(venv) a1234@pi:~/my_project$
```

Step 4: Camera Test

First of all, I had to verify the functioning of the Logitech C270 HD Webcam before object detection could be deployed. The camera feed was tested to capture clear images using Python and OpenCV in real time. The following script was used for capturing a test image and saving it:


```
import cv2
cap = cv2.VideoCapture(0)
ret, frame = cap.read()
cv2.imwrite("test.jpg", frame)
cap.release()
```

This step confirmed the camera's readiness for real-time video input.

Step 5: YOLO Setup

To implement YOLO v3, I cloned the Darknet repository and installed the required dependencies:

```
git clone https://github.com/AlexeyAB/darknet
cd darknet
make
```

I configured the YOLO model for the project by editing the .cfg file to match the number of classes (bottles and cans) and customized anchor boxes. The weights for YOLO v3 were downloaded and initialized for training.

Step 6: Deploy and Integrate

Finally, I trained the YOLO v3 model using the prepared dataset with the following command:

```
./darknet detector train data/obj.data cfg/yolov3.cfg yolov3.weights
```

Then, the final model was deployed on the Raspberry Pi, which further allowed integration with the robotic arm and rover system. The arm sorts bottles and cans effectively guided through its detection.

This is how a structured path should be moved in seamlessly towards YOLOv3 from limitations in YOLO v2 to real-time object detection and classification at high accuracy.

05. Challenges and Solutions

5.1 Technical Challenges

1. Object Detection Accuracy

Challenge: During initial testing, the YOLO v2 model struggled with distinguishing bottles from cans under poor lighting or cluttered backgrounds.

Solution: The YOLO v3 model had to be implemented, which has much better accuracy and performance in the object-detection domain. Further extensions added many more images to increase diversity within the dataset, and the model was fine-tuned in order to optimize chances of detection, regardless of surroundings. Preprocessing by adjusting contrast and normalizing the image further helped to enhance real-time performance for the provision of better detection in harsher environments.

2. Raspberry Pi Real-Time Processing

Challenge: The Raspberry Pi 3A had some initial issues with handling the computational load of the object detection model, which resulted in slower detection speeds.

Solution: The NCS2 accelerates processing through the integration of the Intel Neural Compute Stick 2. It optimizes the system by reducing model complexity—such as using lower resolution for input frames—with a minimal loss of accuracy. The Raspberry Pi 3A had some initial issues with handling the computational load of the object detection model, which resulted in slower detection speeds.

3. Neural Compute Stick 2

It optimizes the system to reduce model complexity—such as using lower resolution for input frames—with minimum loss of accuracy. Hardware-Software Integration.

Challenge: The interaction between Raspberry Pi and Arduino would sometimes have timing problems, which means the robotic arm would respond a little late.

Solution: The communication protocols were refined and further additions of error-handling routines were made to ensure data flowed through seamlessly.

5.2 Non-Technical Challenges

1. Dataset Collection

Challenge: It was challenging to collect and label a sufficient number of images for training and testing.

Solution: The dataset was expanded by combining open-source datasets with manually collected images; then, it was labeled by using tools like LabelImg, which increases the labeling speed.

2. Team Coordination

Challenge: It was challenging to coordinate the activities and maintain consistency in progress among team members, as each had different responsibilities.

Solution: Weekly meetings were held to update progress and resolve issues collaboratively.

06. Project Outcomes and Reflections

6.1 Project Outcomes

The project has successfully attained the main objectives through the design and implementation of the intelligent waste-sorting rover. The following outcomes were realized:

Object Detection and Classification: The YOLO v3 model, working on Raspberry Pi with NCS2, had very good accuracy in the detection and classification of bottles and cans in real time. **Robotic Arm Functionality:** The robotic arm was able to pick up and sort objects into designated bins, showing good integration of hardware and control algorithms.

Rover Mobility: The Arduino-driven forward-wheeled drive by the MG996R servos allowed for fluent navigation and made fine-tuned approaches to an object possible.

Sustainability Goals: This project has shown that intelligent systems have a place in contributing to solving waste management challenges. In that sense, it corresponds well with the UN's Sustainable Development Goal 12: Responsible Consumption and Production.

6.2 Reflections on the Project

The following pages provide informative insights into both technical and non-technical aspects, which have furthered the aim of developing smart systems:

Technical Reflections:

- Dataset diversity can help in understanding the robust detection of objects in the real-world environment. Emphasized also were the preprocessing requirements on the dataset coupled with model tuning for real-world run times.
- System Integration: This happened to be an intricate but rewarding process, which clearly demonstrated the principle of iterative test and effective troubleshooting.

Teamwork Reflection:

Challenges were overcome, and project deadlines were met through effective collaboration and clear communication. Regular team meetings and task management tools greatly facilitated progress.

The division of responsibilities allowed team members to specialize and contribute their expertise, but interdependence required careful coordination.

In case there were any disagreements among the team members, we sorted them out by communicating with each other and by practicing democracy.

6.3 Key Learnings

The project provided hands-on learning experience in the following areas:

- The use of YOLO models for real-time object detection, and the model training and fine-tuning process in detail.
- Hardware-software integration, especially how microcontrollers can be synchronized to higher-level processors (Arduino and Raspberry Pi).
- The value of iterative testing in refining both the hardware and software components of the system.

This project-in addition to reaching the set aims-contributed immensely to key competencies regarding problem-solving and teamwork, in addition to system design. The reflection and learning in the project pave the way for continued work with robotics and intelligent systems.

07. Conclusion

7.1 Summary of the Process

The project designed and developed an intelligent waste-sorting rover for the detection, classification, and sorting of bottles and cans. The work involved a detailed study and selection of suitable technologies, such as Raspberry Pi, Intel NCS2, and YOLO v3 for real-time object detection. Varied data set preparation was done by collecting images and labeling them for training and testing. Designing and assembling the hardware components like servos, robotic arms, and rover chassis was performed. It integrates hardware and software, where YOLO v3 has been fine-tuned to be the fastest and most accurate. A custom sorting algorithm was implemented, together with motion control for precise object manipulation. The iteration of tests ensures that the system performs in all conditions reliably. This project will showcase how integration can be used to solve practical waste management challenges from the perspective of computer vision, robotics, and intelligent systems in a sustainable and ethical manner.

7.2 Future Recommendations

To enhance the project in future iterations, several areas could be improved. Increasing the dataset and exploring more advanced models such as YOLOv5 or YOLOv8 could increase detection accuracy and performance. Upgrading the hardware, such as replacing the Raspberry Pi with a Jetson Nano for faster processing and enhancing the robotic arm with higher-torque servos and sensors for greater precision, would improve overall capabilities. Scalable designs could be adapted to handle greater waste and the addition of conveyor sorting systems, SLAM technology providing navigation in both larger and more dynamically oriented environments. Again, solar panel additions to such a design have several advantages in promoting a greener system with some forms of cleaner power. Field testing in such locations as recycling centers or outdoors could help fine-tune and adapt the system to everyday situations. These improvements will contribute much to the system's efficiency, versatility, and greater impact altogether.



**Maynooth
University**
National University
of Ireland Maynooth